Tartalomjegyzék

01. Alapok
02. Osztály, adattag, metódus
03. Csomagkezelés
04-05. Egységtesztelés: JUnit
06. Tömb, enum, memmap
07. I/O, kivétel
08. `List`, `Set`, `Map`
09. Sablon
10. Öröklődés
11. Interfész, egyenlőség

Programozási nyelvek Java

Gyakorlati feladatsor

ELTE Informatikai Kar

Bevezető

A gyakorlatok az előadáson vett kódokra és koncepciókra támaszkodnak: az ott tárgyaltakat próbálják ki.

Ezért *még azok számára is, akik felmentéssel rendelkeznek az előadás látogatása alól*, **kötelező tisztában lenni az előző hetek előadásain leadott anyaggal**. Ha szükséges, ehhez vissza lehet nézni az előadás felvételeit.

01. Alapok

Első lépések

1. demó feladat

Töltsd le az előadások demóinak `zip` fájlját, és csomagold ki egy külön könyvtárba. Futtasd az első néhány programot.

- Használd a `cmd` és a `pwsh` (PowerShell) parancssort is legalább egyszer.
- Használd a terminált külön programként és a VS Code-ba integráltan is.
  - Kezeljük egyszerűen a fájlokat: ne legyen külön fordítás, csak futtasd őket így: `javac TheoryDemoProgram.java`

Feladatok

1. feladat

Felhasznált alapok: `BasicHelloWorld{N}` , `InputWithIO{1}` . Adott egy szöveg a `txt` változóban. A `SimpleTextManipulation.java` főprogramja hívon rá sorban a `main01` , `main02` stb. segédmetódusokra `txt` átadásával. Ezek működése a következő legyen.

1. Írd ki a hosszát.
   - Az API dokumentációban a megfelelő típusban keress rá egy itt jól alkalmazható műveletre.
     - A dokumentáció linkje megtalálható a példakódokban.
   - A további feladatrészekben is lehet, hogy utána kell majd nézned bizonyos metódusoknak. Ha körbenéztél, de nem találod, kérd a gyakorlatvezető segítségét.
2. Írd ki egy-egy sorban az első és az utolsó karakterét.
3. Most az első és az utolsó karaktert tedd össze egy szövegbe, és úgy írd ki.
   - Vajon miért lett rossz a kiírás?
   - Javítsd meg.
4. Most ne csak egy-egy betűt vegyél fel, hanem `n` -et elölről és hátulról is. Ez legyen paraméter.
   - Ehhez persze az kell, hogy legyen legalább `n` karakter benne.
     - Nézd meg a program kimenetét, ha nincsen benne elég karakter.
   - Innentől csendben feltételezzük minden feladatban, hogy ha a leírás nem említi külön, akkor minden bemenő adat érvényes.
5. A metódus ne kiírja, hanem adja vissza a szöveget.
   - A megírt kódot mindig ki szeretnénk próbálni. Most és a továbbiakban is, ha a feladat külön nem jelzi, hogyan kell ezt megtenni, akkor a kapott eredményt a `main` írja ki.
6. A metódus rakja össze a szöveg összes prefixét egy nagy szövegbe, és ezzel térjen vissza.
   - Példa: az `abcde` szövegből ezt alakítsa ki: `aababcabcdabcde` .

2. feladat

Felhasznált alapok: `InputWithIO{2,3}` . A `FindInTextBefore.java` fájlban `findFirstIdx` egy szöveget és abban két pozíciót kap meg. Adja vissza, hogy melyik indexen található a kijelölt részszöveg első másolata.

1. Ennek egy variánsa (`findFirstIdxIgnoreSpaces`): hagyd figyelmen kívül a szóközöket. - Tipp: a szóközöket cseréld le üres szövegre. - Tipp: ne írd meg újra az egész kódot, hanem használd az előző verziót: hívj át rá.
2. `findFirstIdxIgnoreCase` : a szóközök mellett a kis- és nagybetűk különbségét se tekintsd.
   - Tipp: a vizsgálatban úgy dolgozz a szöveggel, hogy kisbetűsre konvertálod.
3. Mindegyik fentihez írj egy `verifyMMM` metódust (pl. `verifyFindFirstIdxIgnoreCase`), ami a társával megegyező paraméterezésű, és áthív rá. A kapott eredményt viszont nem arra használja fel, hogy kiírja azt, hanem ellenőrzi, hogy helyes-e az eredmény: a kapott pozíción tényleg a megadott szöveg található-e. Ezzel mint logikai értékkel tér vissza.
   - Amikor programrész működését ellenőrzöd, minden jellemző, érdemileg különböző esetet próbálj ki egyszer.
   - Itt kettő lehetőség van: vagy szerepel a szöveg korábban is, vagy az első megtalált index maga a szöveg.

- Azt továbbra is feltételezzük, hogy minden bemenetünk helyes, és adható hozzá helyes kimenet.

3. feladat

Felhasznált alapok: `InputWithIO{4,5}` . A `LaundryList` főprogramja olyan paraméterezésű, hogy a szavakból egy mosási lista áll össze, pl. így: `3` , `trousers` , `6` , `pants` , `9` , `T-shirts` , `100 neckties` . Az alábbi metódusoknak ezt egyszerűen át kell adni.

1. A `hasAllGoodNames` adja meg, nem rontotta-e el a bemenetet, aki átadja: mindegyik névnek van-e tartalma, vagy van-e köztük üres szöveg.
   - A szövegeknek van egy "ennek a vizsgálatára szakosodott" metódusa, a metódus megírandó a felhasználásával és anélkül is.
2. A `getItemCount` adja meg, összesen hány ruha szerepel. (A példában `4` .)
3. A `getKindCount` adja meg, hányfajta ruhadarab szerepel a leírásban. (A példában `4` .)
   - A ruhadarabok fajtáinak érintésénél ne számold azokat, amelyek még később is szerepelnek.
4. A `getCount` kap egy külön, első paramétert: egy ruhafajta nevét. Azt kell megadnia számként, hogy a ruhafajtából hány darab van a mosási listán.
   - Hogyan kell megírni a kódot, ha a ruhafajta biztosan legfeljebb egyszer szerepelhet?
   - Hogyan kell megírni a kódot, ha a ruhafajta többször is szerepelhet?
5. A `hasDuplicate` adja meg, hogy szerepel-e bármelyik ruhafajta kétszer is. A megoldásnak nem kell szörnyen hatékonynak lennie.

02. Osztály, adattag, metódus

1. feladat

a. Hozd létre egy `Point` osztályt, amely két mezőt tartalmaz: `x` és `y` típusa `double` .

Készíts hozzá egy egyszerű konstruktort, amely két `double` paramétert kap, és ezek alapján állítja be a mezőket.

Készíts egy `print()` metódust, amely a következő formában írja ki az értéket a sztenderd kimenetre: `At (10.0, 20.0)` .

- Használj `.formatted` -et a kiíráshoz a `%2.1f` formátumvezérlőt.
- Az áttekintő fóliasorban található példa a `.formatted` használatára.

Készítsd el a `PointMain.java` fájlt, amiben a `main` hozzon létre egy pontot, és írassa ki.

- A `main` elején próbáld ki a következő hívásokat is. Változik-e a kiírás? - `Locale.setDefault(Locale.of("en"));` -

`Locale.setDefault(Locale.of("hu"));`

- Az alábbi példák részint az egyik, részint a másik beállítással készültek.

b. Készíts egy `shift(dx,dy)` metódust, amely a pont koordinátáit eltolja `dx` és `dy` értékkel.

- Itt a `d` azt jelzi, hogy ezek a paraméterek különbségeket, „deltákat” jelentenek.

```
At (10.0, 20.0)
At (11.2, 16.6) after being shifted by (1.2, -3.4)
```

A hosszabb kiírás megvalósításához készíts egy másik `print()` változatot (a régit tartsd meg!), amelyik két paramétert kap: egy szöveget és egy `Point` objektumot. A második kiírást ennek felhasználásával készítsd el a következő séma alapján:
`At <aktuális pont> <paraméter szöveg> <paraméter pont>`

c. Most a `PointMain` `main` függvénye kapjon paramétereket is, szövegeket: `String... args`.

Közülük az első kettő írja le a kezdőpontot. A továbbiak kettesével értelmezendők: mindegyik egy-egy eltolást ír le.

- Használj `Double.parseDouble` -t a szövegek átalakításához.
- Írasd ki mindegyik változást.
- Példa: ha a bemenet `10.0 20.0 1.2 -3.4 10.0 10.0 -5.11 7.77 -4.321 1.234`, akkor a kimenet:

```
At (10.0, 20.0)
At (30.0, 21.2) at step #1 after being shifted by
        (20.0, 1.2)
At (26.6, 31.2) at step #2 after being shifted by
        (-3.4, 10.0)
At (36.6, 26.1) at step #3 after being shifted by
        (10.0, -5.1)
At (44.4, 21.8) at step #4 after being shifted by
        (7.8, -4.3)
```

Próbáld ki a programot `jshell` -ből és parancssorból is.

d. A `Point` ok létrehozásának megkönnyítése céljából hozz létre egy konstruktort, ami megkapja az egész `args` ot egyben és még egy `idx` indexet is. A pontot az `idx` indexen lévő (`x` koordináta) és a `idx+1` indexen lévő (`y` koordináta) számokból állítsa elő.

Használd ezt az előző `main` egyszerűsítésére.

e. A `Point` ban hozz létre egy `mirror(cx,cy)` metódust, ami a pontot tükrözi a megadott `cx` és `cy` koordinátákra.

- Itt a `c` azt jelzi, hogy ezek a paraméterek a tükrözés „középpontját” adják meg.

Készíts egy `PointMain` programot, ami kipróbálja a `Point` osztályt, és a következő lépések szerint írja ki az eredményt.

```
At (10.0, 20.0)
At (11.2, 16.6) after being shifted by (1.2, -3.4)
At (8.8, 3.4) after being mirrored on (10.0, 10.0)
```

f. Ha a főprogram nem kap egyetlen paramétert sem, akkor állítsd az `args` változó tartalmát a következőre: `"10.0 20.0 1.2 -3.4 10.0 10.0".split(" ")` , és azt felhasználva fusson tovább a program.

g. Ha csak a kezdőpont van megadva (az első két paraméterben), akkor a program olvassa a bemenetet a sztenderd bemenetről az `IO.readln` segítségével.

Ezután olvassa be egy eltolás és egy tükrözés adatait: összesen négy koordinátát.

- Ha az első szám, amit a felhasználó ad, egy speciális érték (pl. `-123456`), akkor a program fejezze be a futást.
  - Adj nevet ennek a konstansnak: `END_INDICATOR` .
  - „Érdekes” számokat név nélkül használni rossz gyakorlat, gúnynevük magic number.
- Írj ki hasonló üzeneteket, mint az előző feladatban.
- Amíg a speciális érték meg nem érkezik, olvass be és hajts végre egy eltolást, egy tükrözést, egy eltolást, egy tükrözést stb.

h. Bónusz feladat: tudsz-e olyan programot készíteni, ami minden eddigi lehetőséget tartalmaz?

- Tehát ha nincsen paraméter, alapértelmezett készlettel indít; ha csak egy pont van megadva, a többit folyamatosan beolvassa; ha minden meg van előre adva, azokkal dolgozik.
- Készíts egy segédfüggvényt `getNextPoint` néven, amely a következő pontot az argumentumokból vagy a sztenderd bemenetről adja vissza, ahogy szükséges.

```
At (10,0, 20,0)
Input x for shifting Point#1: 11.1
Input y for shifting Point#1: 22.2
At (21,1, 42,2) at step #1 after being shifted by (11,1,
Input x for mirroring Point#1: 33.3
Input y for mirroring Point#1: 44.4
At (45,5, 46,6) at step #2 after being mirrored on (33,3
Input x for shifting Point#2: -123456
Input y for shifting Point#2: 0
Special input encountered, exiting
```

i. Készíts egy `distance(p)` metódust, amely kiszámítja a távolságot az aktuális pont és a paraméterként megadott `p` pont között.

- Használd a `Math.sqrt(...)` függvényt és a Pitagorasz-tételt.
- Készíts egy `PointPath.java` fájlt, amelynek `main` függvénye páros számú argumentumot kap. Minden két érték egy-egy pontot ír le; feltételezheted, hogy kettő vagy több van. A `distance` felhasználásával írasd ki a következőt.

```
At (10,0, 20,0)
At (1,2, -3,4) with a total distance of 25,0 and a
        distance of 25,0 from (1,2, -3,4)
At (10,0, 10,0) with a total distance of 41,0 and a
        distance of 16,0 from (10,0, 10,0)
At (-5,1, 7,8) with a total distance of 56,3 and a
        distance of 15,3 from (-5,1, 7,8)
At (-4,3, 1,2) with a total distance of 62,9 and a
        distance of 6,6 from (-4,3, 1,2)
```

2. feladat

Készítsd el a `Complex` osztályt, ami egy komplex számot reprezentál valós és képzetes résszel. Adattagjai: `re` és `im`, `double` típusúak.

Készítsd el a következő metódusokat.

- `abs()` : kiszámítja a komplex szám abszolút értékét: re^2+im^2 négyzetgyöke.
  - Tipp: érdemes megnézni a `Math` osztály dokumentációját.
- `add(c)`, `sub(c)`, `mul(c)` : összeadja/kivonja/szorozza a számot a `c` paraméterrel.
  - A kivonást valósítsd meg úgy, hogy meghívod az `add` metódust a negált `c` -vel.
  - A szorzás képlete: `(a + ib) * (c + id) = (ac - bd) + (ad + bc)i` .
- `getText` : visszaad egy szöveget, amely a komplex számot a következő formában mutatja: `4,0+2,0i` vagy `1,0-13,0i` , vagy `3,7` , ha a képzetes rész nulla.
- `print` : két változatban, hasonlóan, ahogy a `Point` esetén.

A `main` -ben végezz néhány műveletet, és írasd ki az eredményeket például így.

```
The number is 4,0+2,0i
The number is 5,0-11,0i after adding 1,0-13,0i
The number is 16,1-8,6i after subtracting 11,1+2,4i
The number is 12,8-7,4i after adding -3,3+1,2i
```

Gyakorló feladatok

1. gyakorló feladat

Készítsük el a Complex osztályba a `conjugate()` műveletet, mely a komplex számot átalakítja a komplex konjugáltjára. Készítsük el a `reciprocate()` metódust, mely a komplex számot reciprokára alakítja. Definiáljuk a `div(c)` művelet, mely elosztja a komplex számot a paraméterként kapott `c` komplex számmal.

2. gyakorló feladat

Készítse el a `Line` osztályt, mellyel egy adott sík egyeneseit reprezentálhatjuk. Egy egyenest az $ax + by = c$ összefüggés ír le, ahol `a` , `b` és `c` számok `double` típusúak. (Ezek lesznek az osztály adattagjai.)

Írjon az osztályba egy `contains(p)` műveletet, mely eldönti, hogy egy `p` pont rajta van-e az egyenesen!

Írjon egy `isParallelWith(l)` és egy `isOrthogonalTo(l)` metódust, melyek eldöntik, hogy az egyenes párhuzamos-e a paraméterként kapott `l` egyenessel, illetve merőleges-e rá!

3. gyakorló feladat

Készítse el a `Segment` osztályt, mely egy szakaszt reprezentál. A szakasz objektumok ábrázolásához a két végpont koordinátáit tároljuk el. Az adattagok `x1` , `y1` , `x2` , `y2` legyenek, mind `double` típusú.

Írjon az osztályba egy `line()` metódust, mely visszaad egy olyan `Line` objektumot, amely a szakaszra illeszkedő egyenest reprezentál.

Írjon az osztályba egy `contains(p)` műveletet, mely eldönti, hogy egy `p` pont rajta van-e a szakaszon!

Készítsen `orientation(p)` metódust a `Segment` osztályba, mely eldönti, hogy a szakasz kezdőpontjából a végpontjába mutató vektor, valamint a szakasz végpontjából a paraméterként kapott `p` pontba mutató vektor milyen orientációjú. A metódus adjon vissza 0-t,

ha a `p` rajta van a szakasz által meghatározott egyenesen, adjon vissza pozitív értéket, ha a két vektor az óramutató járásával megegyező irányban van egymással, illetve negatív értéket, ha az óramutató járásával ellenkező irányú. Ez elég egyszerű: ha a `p` pont koordinátáit x_3 és y_3 jelöli, akkor a metódus az alábbi kifejezést adja vissza.

$$(y_2 - y_1)(x_3 - x_2) - (y_3 - y_2)(x_2 - x_1)$$

Készítsen egy `intersects(s)` metódust, mely visszaadja, hogy a szakasznak van-e közös pontja a paraméterként kapott `s` szakasszal! A megoldáshoz használja az alábbi segítséget!

http://www.dcs.gla.ac.uk/~pat/52233/slides/Geometry1x1.pdf

4. gyakorló feladat

a

Készítsük el a `Rectangle` osztályt, mely a koordinátatengelyekkel párhuzamos oldalú téglalapok ábrázolására alkalmas! A `Rectangle` objektumukban tároljuk el valamelyik csúcspontjuk `x` és `y` koordinátáját, valamint a téglalap szélességét és magasságát. Ez négy adattagot jelent: `x`, `y`, `width` és `height`. Legyen mindegyik típusa dupla-pontosságú lebegőpontos típus.

A szélesség és a magasság felvehet negatív értéket is. Legyen például az `r` négy adattagja rendre 1, 5, 6, -2. Ekkor az `r` bal alsó csúcsának koordinátái 1 és 3 lesznek.

Definiáljuk a `Rectangle` objektumokon a `topLeft()`, a `topRight()`, a `bottomLeft()` és a `bottomRight()` metódusokat, melyek mindegyike egy `Point` objektumot ad vissza, értelemszerűen a téglalap megfelelő csúcsának a koordinátáit.

A `Rectangle` osztályhoz készítsünk egy főprogramot, mely meghatározza a parancssori argumentumaként kapott téglalapok befoglaló téglalapjának csúcspontjait. A főprogram parancssori argumentumai számok legyenek (legalább 4). Minden számnégyes egy `Rectangle` objektumot határoz meg. Ezeket kell feldolgozni, és a végén kiírni a befoglaló téglalap bal alsó és jobb felső csúcsainak koordinátáit.

```
$ java RectangleMain 3 5 1 -7 2 5 8 8
Bounding rectangle: 2.0;-2.0 - 10.0;13.0
```

A fenti példa két téglalap befoglaló téglalapját határozza meg. Az első téglalap egyik csúcsa (3,5) koordinátájú, szélessége 1, magassága -7. A másik téglalap egyik csúcsa (2,5) koordinátájú, szélessége és magassága egyaránt 8. Az eredményként kapott befoglaló téglalap bal alsó csúcsa (2,-2), jobb felső csúcsa (10,13), amely egyébként a második kapott téglalap jobb felső csúcsa is.

Segítség: a befoglaló téglalap bal alsó csúcsának x-koordinátájának meghatározásához keressük meg a legkisebb értéket a kapott téglalapok bal alsó csúcsának x-koordinátái között, stb.

b

Vegyük észre, hogy a `Double.max(,)` és `Double.min(,)` függvényekkel kényelmese(bbe)n tudjuk megoldani a fenti feladatot, mint elágazásokkal és "?:"-kifejezésekkel!

03. Csomagkezelés

Demo

1. demó

Szervezze az előző órai `Point` osztályt és az azt bemutató `PointMain` főprogramot a `point2d` csomagba.

1. A `Point` osztály ne látszódjon ki a csomagból: ne legyen publikus.
2. Készítse el a `pointless.AnotherMain` osztályt. Ez az osztály teljesen minősített neve; azt jelzi, hogy a `pointless` csomag `AnotherMain` osztálya lesz. A megfelelő fájl: `pointless/AnotherMain.java`.
   - Próbálja példányosítani a `Point` osztályt az `AnotherMain` osztályból. Figyelje meg a hibaüzenetet.

Fontos részletek.

- Eddig "kompakt forráskódokat" írtunk, de ezek *nem használhatnak* csomagokat. Ezért innentől teljesen kiírt osztályokat kell készítenünk, és a kód bőbeszédűbb lesz.
- A csomagoknevek nevének és a könyvtáraknak *megegyező szerkezetűnek* kell lennie.
- A fordítást az *alapkönyvtárból* (ahonnan a könyvtárszerkezet indul) kell indítani.
  - Ha a terminál másik könyvtárra van állítva, a fordítás sikertelen lesz!

Feladatok

1. feladat

Kísérletezzünk egy kicsit az osztályok láthatóságával. A `Point` osztály váljon láthatóvá más csomagokból is.

a. Az `AnotherMain` most már valóban készítsen `Point` példányokat, két módon.
   i. A teljesen minősített neve használatával.
   ii. Egy `import` segítségével.
b. Készítse el a `PointMainInDefaultPackage` osztályt a névtelen csomagban.
   - Ebből használható-e a `Point`: készíthető-e `Point` példány a `PointMainInDefaultPackage` osztályban?

- Megfordítva: ez az osztály használható-e a `Point` osztályból?

2. feladat

Készítsen `plane.PublicCircle` osztályt, amellyel egy síkbeli kört reprezentálunk. Egy körnek van középpontja, `double` típusú `x`, `y` adattagja, amelyeket inicializáljunk `0.0`-ra, illetve `double` típusú sugara (`radius`), amelyet inicializáljunk `1.0`-re. Az adattagok legyenek publikusak benne.

a. Írjon `getArea()` metódust, amely ismét a kör területével térjen vissza.
   i. A `plane.but.not.flying.CircleMain.main` metódus a következőket tegye.
      - Példányosítson egy `PublicCircle` objektumot.
      - Írja ki a területét.
      - Állítsa be a középpontját `(5, 2)`-re, sugarát `10.0`-re.
      - Írja ki újra a területét.
b. Készítsen az előzőhöz hasonló `plane.Circle` osztályt, de ebben az adattagokhoz csak megfelelően megírt getter és setter metódus férhessen hozzá.
   i. Az utóbbi váltson ki egy `IllegalArgumentException` kivételt, ha a sugara `0.0` vagy negatív szám lenne.
   ii. A `CircleMain.main` próbálja ki az új funkcionalitást.
      - A kivételt nem kell elkapni, azzal az anyag későbbi részén foglalkozunk.
c. A `Circle` osztályhoz készítsünk konstruktort, az adattagok beállítását ezzel végezzük.
   i. A konstruktor a setterhez hasonlóan válthasson ki kivételt.
   ii. A `CircleMain.main` próbálja ki az új funkcionalitást.

3. feladat

Készítsen `string.utils.IterLetter` osztályt.

a. Ennek a konstruktora `String` paraméterű.
   - Ha ez `null`, váltson ki `IllegalArgumentException` kivételt.
b. Az osztálynak legyen egy `printNext()` metódusa.
   - Ez egy új sorban írja ki a szöveg első karakterét, majd újabb hívásra a második karakterét és így tovább.
   - Ha a szöveg összes karakterét kiírtuk a képernyőre, akkor a metódus egy üres sort ír ki minden további hívásra.
c. Készítse el a következő, főprogramot tartalmazó osztályokat: `Main` (a névtelen csomagban), `stringmain.Main`, `string.main.Main`, `string.utils.Main` és `string.utils.main.Main`.
   - Ezek közül egy kivételével mindegyiknek az a feladata, hogy áthívjon egy másik `main`re. Tehát egy négy hosszú híváslánc alakul ki.
      - A sorrend szabadon megválasztható.
      - Tipp: az `X` osztály `y` (osztályszintű) metódusát így lehet meghívni: `X.y(...)`.
         - Milyen formában kell most `X` nevét használni?

        - Hányfajta sorrend létezik?
        - Néhány sorrend érvénytelen. Melyek? Miért?
    - Az utoljára érintett főprogram két parancssori paramétert kap meg. A kód a `printNext` metódust annyiszor hívja meg a második paraméter szövegével, amennyi az első paraméterben kapott szám értéke.
    - Futtassa mindegyik főprogramot. Paraméterként válasszon különféle szövegeket, az üreset is (ez `""` alakban adható át).
d. Az `IterLetter` `reset()` metódusa újrakezdi a szöveg végigjárását.
    - Tehát a soron következő `printNext` hívások az első karaktertől folytatják a kiírásokat.
    - A főprogramok egyszer hívják meg ezt a metódust `m` kiírás után, és még `k` kiírást tegyenek meg. Itt `m` és `k` a harmadik és negyedik parancssori paraméter.
e. Legyen az osztályban egy `hasNext()` metódus is. Ez pontosan akkor tér vissza igaz értékkel, ha van még kiírható betű.
    - A főprogramok hívják ezt meg az újrakezdés előtt és után, valamint közvetlenül a program befejeződése előtt.

Gyakorló feladatok

1. gyakorló feladat

A `zoo.animal.Panda` osztály egy pandát reprezentál.

a. Az osztály a panda nevének, életkorának és tartózkodási országának eltárolására alkalmas. Ehhez értelemszerű típusú és nevű adattagokat kell használni.
b. Az osztálynak két konstruktora van.
    i. Az első egy újszülött pandát ír le: megkapja és eltárolja a másik két adatát.
    ii. A másik konstruktor a név kivételével kapja meg az adatokat. Ennek a pandának az legyen a neve, hogy `Y years old foundling from C`, ahol `Y` és `C` helyén az életkor és az ország szerepel.
c. Az osztály rendelkezik egy `happyBirthday()` metódussal is. Ez kiírja a panda nevét, országát és az eggyel megnövekedett életkorát is. A metódus egy számot is kap paraméterül (`limitYear`), amennyiben az életkor ezt meghaladja, akkor a pandát visszaköltöztetik a Kínai Népköztársaságba.
d. A `zoo.keeper.Crikey` főprogramja próbálja ki a fentieket.

2. gyakorló feladat

Bővítse a `string.utils` csomagot a `string.utils.IterWord` osztállyal.

a. Az osztály konstruáláskor egy szöveget kap meg.
b. Az osztály `printNext()` metódusa új sorban a képernyőre írja a szöveg következő szavát.
c. Az osztálynak szintén legyen `reset()` és `hasNext()` metódusa.
d. A már létező főprogramok mellé kerüljön egy-egy `WordMain`, amely bemutatja az osztály használatát.

3. gyakorló feladat

Készítse el a `magic.library.Incantation` osztályt, amely egy varázslatos ráolvasást ábrázol. Ez egy `text` szöveges és egy `index` egész szám adatot tárol.

a. Az osztály egyik konstruktora átveszi és beállítja a két paraméter értékét.
   - Ha az előbbi `null`, váltson ki `IllegalArgumentException` kivételt.
b. A másik konstruktor egy `Incantation` példányt kap meg, és ennek az adattagjaiból tölti fel a sajátokat.
c. Legyen mindkét adattagnak gettere, az `index` nek settere is.
d. Az osztály `enchant()` metódusa egy `Incantation` példányt (`otherInc`) kap meg és egy `isPrepend` logikai értéket. A metódus az alábbiak szerint módosítja a hívott példány szövegét.
   i. Először megpróbáljuk felvenni a `text` szöveg `index` edik szavát.
      - Ha például `index` értéke `3`, akkor a harmadik szóról van szó.
      - Tipp: a `String` osztály `split` metódusát érdemes használni.
   ii. Ha nincsen ilyen szó, mert `index` túl magas vagy alacsony (akár negatív is lehet), a metódus hamis visszatérési értéket ad.
   iii. A felvett szót `otherInc` szövege elé írjuk, ha `isPrepend` értéke igaz, és mögé, ha hamis.
      - Egy szóköz is kerüljön a régi szöveg és az új szó közé.
   iv. Az `index` adattag eggyel megnő/lecsökken `isPrepend` értékétől függően.
   v. A metódus igazzal tér vissza, jelezve, hogy a szöveg megváltozott.
e. A `magic.Soliloquy` osztály `reciteIncantations()` metódusát a főprogram fogja hívni, és a következőket teszi.
   i. Két `Incantation` példányt kap meg (`inc1` és `inc2`), valamint egy `idx` egész és egy `startWithAppend` logikai értéket.
   ii. Háromszor hívja meg az `inc1` példányra az `enchant` metódust. Az első paraméter mindegyik esetben `inc2`.
      - A második paraméter az első hívásban `startWithAppend` ellentéte.
      - A másodikban `startWithAppend`.
      - Harmadjára `true`.
   iii. Mindegyik hívás után írja ki a kifejezés visszatérési értékét és mindkét objektum adatait.
      - Ez utóbbiakat a getterek segítségével lehet elérni.
      - A formátumhoz lásd a lenti példát.
      - A kiírás kódja a `printStatus` segédfüggvénybe kerüljön.
   iv. Az első kiírás után `inc1` `index` adattagjának értéke álljon `idx` re.
f. A `magic.Soliloquy` osztályban levő főprogram hat parancssori paramétert vár (`argN` lentebb) és a következőket teszi.
   i. Feltehető, hogy a paraméterek száma és tartalma megfelelő, nem kell ellenőrizni.
   ii. A főprogram elkészíti az `inc1`, `inc2` és `inc3` `Incantation` példányokat. Az első kettő tartalmát az első négy parancssori paraméter értékei alapján inicializálja, a harmadikat pedig a másodikból.
      - Az első és harmadik parancssori paramétert idézőjellel (`"`) kell majd körbe-

venni, mert szóközöket tartalmaznak.

iii. Ezután a következőket teszi.
   - Meghívja a `reciteIncantations` segédfüggvényt az `inc1, inc2, arg5, true` paraméterekkel.
   - Beállítja `inc1 index` ét `arg6` értékre.
   - Meghívja a `reciteIncantations` segédfüggvényt az `inc1, inc3, arg5, false` paraméterekkel.

g. Kipróbálás: a főprogram kapja meg a következő paramétereket: `programming is a creative activity`, `4`, `to be or not to be`, `-123`, `1`. Ennek hatására a kiírások legyenek az alábbiak.

```
true;7;programming is a fun and creative activity;-123;t
true;1;programming is a fun and creative activity;-123;t
true;0;programming is a fun and creative activity;-123;p
false;0;programming is a fun and creative activity;-123;

true;3;programming is a fun and creative activity;-123;f
true;1;programming is a fun and creative activity;-123;f
true;2;programming is a fun and creative activity;-123;f
true;1;programming is a fun and creative activity;-123;i
```

4. gyakorló feladat

Készítsen egy `game.utils.Vehicle` osztályt, amellyel egy játék járművét reprezentáljuk. Egy járműnek van modelid-je (`int`), rendszáma (`String`), és két színállapota (`color1`, `color2` `int` típusú adatok). A rendszámhoz készítsen setter és getter metódusokat.

Készítsen `game.Player` osztályt, amellyel egy játékost reprezentálunk. Egy játékosnak van neve (`String`), IP-címe (`String`), egészségi állapota (`int`) és lehet járműve (`Vehicle`) (ha nincsen, akkor tároljunk `null` értéket).

A játékos osztályhoz készítsen `print()` metódust, amellyel ki lehet íratni egy játékos legfontosabb információit: nevét, IP-címét, egészségi állapotát, illetve járművének rendszámát (ha van).

Készítsen `game.Main` főprogramot, amelyben példányosít legalább 3 járművet és legalább 2 játékost. Az egyik játékoshoz tartozzon jármű. A főprogram írja ki a képernyőre a játékosok adatait.

04-05. Egységtesztelés: JUnit

Demo

1. demó

Írjunk `fib()` metódust a `famous.sequence.Fibonacci` osztályba, amely egy `n` számot kap meg, és kiadja az `n` -edik Fibonacci-számot. Feltesszük, hogy az `n` bemenet kis, pozitív szám.

Teszteljük a következőképpen.

- A metódus kapjon `static` minősítőt. Erről a kulcsszóról részletesebben később lesz szó.
- A mellékelt `famous.sequence.FibonacciStructureTest` használata.
  - Ezt a csomagjának megfelelő könyvtárba kell elhelyezni.
  - Ez a `CheckThat` eszközt használja, aminek szintén elérhetőnek kell lennie.
  - Fordítás és futtatás. A parancsokat lásd a demó `zip` fájlban.
    - Először a `FibonacciStructureTest` osztályt fordítsuk.
    - Most a `Fibonacci.java` osztályt külön paranccsal kell lefordítani, mert a tesztelő nem hivatkozik rá közvetlenül, és így az előző fordításban a `javac` nem keresi meg.
    - Futtassuk a teszteket.
- Készítsünk `famous.sequence.FibonacciTest` tesztelőt.
  - Ebben legyen teszt néhány konkrét `n` értékre.
  - Próbáljuk ki a paraméterezett tesztelést is.
  - Fordítás és futtatás.
    - A folyamat jobb megértése érdekében töröljük le a korábban elkészült összes `.class` fájlt. (A `.java` fájlok maradjanak meg!)
    - Fordítsuk le a `FibonacciTest` kódját. Ez a `Fibonacci.java` fájlt is megtalálja és lefordítja.
    - Futtassuk a teszteket.
- A mellékelt `famous.sequence.FibonacciTestSuite` használata.
  - Ezt is a csomagjának megfelelő könyvtárba kell elhelyezni.
  - Ismét töröljük le a korábban elkészült összes `.class` fájlt.
  - Fordítsuk le a `FibonacciTestSuite` kódját. Ezúttal `Fibonacci.java` és a két korábbi tesztelő is lefordul.
  - Futtassuk a teszteket.

Feladatok

1. feladat

Készítsünk háromszögszámokat kiszámító `static famous.sequence.TriangularNumbers.getTriangularNumber()` metódust.

- A tesztelő kód a `famous.sequence.TriangularNumbersTest` osztályba kerüljön.
- Ehhez is jár mellékelt strukturális tesztelő és suite is, ezeknek is rendben le kell futniuk.
  - A fájlokat a megfelelő könyvtárba kell elhelyezni, különben nem működnek.

Kipróbálandók a következő bemenetek.

- nulla
- egy
- a szám, ami a fiatal Gauss anekdotájában szerepel
- mínusz egy
- más negatív szám

A tesztelőbe írjunk szándékosan egy rossz tesztesetet is, direkt elrontott elvárt eredménnyel. Vizsgáljuk meg a futtatás eredményét, benne az összefoglalóval és a *stack trace* részleteivel.

Az osztályba kerüljön `getTriangularNumberAlternative()` is. Ez a képlettel dolgozzon, és adja ugyanazt a kimenetet, mint `getTriangularNumber()` .

- A tesztelő próbálja ezt is ki.

2. feladat

Készítsük el a `static math.operation.safe.Increment.increment()` metódust.

- Legyen `static` minősítőjű.
- Bemenete egy `int` .
- Ha a bemenete a legnagyobb ábrázolható egész, azzal tér vissza, különben a bemeneténél eggyel nagyobb egésszel.
  - Erre így lehet hivatkozni: `Integer.A_MEGFELELŐ_ADATTAG_NEVE` .
  - A keresett név a Java API dokumentációból olvasható ki.

Ugyanebbe a csomagba készítsük el az `IncrementTest` osztályt az `increment()` működésének kipróbálására.

- Bemenete legyen nulla.
- Bemenete legyen a legkisebb ábrázolható egész.
- Bemenete legyen a legnagyobb ábrázolható egész.
- Bemenete legyen egy közepesen nagy, pozitív egész.
- Bemenete legyen egy közepesen nagy, negatív egész.
- Bemenete legyen `-1` .

Készítsünk egy hasonló tesztelő fájlt egy másik csomagba is, és próbáljuk ki ott is.

3. feladat

Készítsük el a mellékelt strukturális tesztelőben (`RecordLabelStructureTest` , `ArtistStructureTest` , `FanStructureTest`) leírt osztályokat a megadott szerkezettel.

Ebben a feladatban előre meg van adva néhány funkcionális teszt is. Ezeket a fájlokat bővítsük az alábbi tesztekkel.

- Tesztelendő, hogy létrehozás után mindegyik objektumra helyes értéket adnak a getterek.
  - A rajongónak, a rajongó művészének, és a rajongó művésze kiadójának a neve

megfelelő kell, hogy legyen.
  - A rajongó létrehozás után közvetlenül még nem költött el pénzt.

A funkcionális tesztelőben az alábbi tesztek érhetők el a `Fan` osztályhoz. Itt a feladat a megvalósítás elkészítése.

- `buyMerchandise()` kiszámítja a rajongó által vásárolt termék árát.
  - A rajongó természetesen csak a kedvenc művészétől vesz termékeket.
  - Az alapár paraméterként van megadva, de ha a rajongó másokkal együtt veszi meg a terméket, az a darabárat fejenként 10%-kal, de legfeljebb 20%-kal csökkenti.
    - A további rajongókat deklaráljuk egy *vararg* paraméterben, így: `Fan... friends`.
    - Így hívható meg a metódus: `fan1.buyMerchandise(100, friend1, friend2, friend3)`, ahol mindegyik `friendN` egy `Fan` példány.
    - A költségcsökkentés mértéke akkor sem haladja meg a 20%-ot, ha kettőnél több extra rajongó csatlakozik.
  - A metódus visszatérési értéke a számított darabár.
    - Ezt az összeget elkölti a rajongó és mindegyik barátja.
    - Az összes költés felét megkapja a kiadó a `gotIncome()` segítségével.
- `favesAtSameLabel()` azt vizsgálja meg, hogy ugyanaz a művész-e két rajongó kedvence.
- Az alábbi négy metódus állítsa elő a rajongó szöveges reprezentációját. Ezek visszatérési értéke megegyezik, de más-más módon állítják elő azt.
  - A `toString1()` metódus a `+` operátorral fűz össze szövegrészleteket.
    - Ez hatékony, de macerás megírni és olvasni.
  - A `toString2()` metódus a `formatted` metódust használja ilyen módon: `"%s=%d".formatted("one hundred", 100)`.
    - Ezt könnyű olvasni.
  - A `toString3()` metódus ezt használja: `String.format("%s=%d", "one hundred", 100)`.
    - Ez az előzőhöz hasonló, kicsit régebbi stílusú.
  - A `toString4()` metódus egy `StringBuilder` példányt használ.
    - A `sb.append(123)` hívással lehet bővíteni a tartalmat.
    - A `sb.toString()` hívással lehet szövegként kinyerni a tartalmat.
    - Ez akkor hasznos, ha a kimenet hosszú és összetett.

4. feladat

A `math.operation.textual.Adder.addAsText()` metódus két `String` paramétert vár.

- Ha mindkettő értelmezhető egész számként, az eredmény az összegük szövegesen.
- Hasonlóan működik, ha a bemenetek értelmezhetőek lebegőpontos számként.
- Ha a bemenetek nem szám jellegűek, az eredmény a következő szöveg: `Operands are not numbers`.
- Az implementáció forráskódja **nem** elérhető, csak a `.class` fájl.
  - Ezt a megfelelő könyvtárba kell tenni.

Ugyanebbe a csomagba készítsük el az `AdderTest` osztályt, ami a következőket próbálja ki.

- `wrongInput` : legalább az egyik paraméter nem szöveges
- `addZero` : egy számhoz nullát adunk hozzá
  - Mindkét oldal lehessen a nulla.
  - Kipróbálandó: a nulla vagy a szám lebegőpontos.
- `add` : összeadunk két számot
  - Az összeadás mindkét sorrendben ugyanazt az eredményt adja.
  - Kipróbálandó: mindkét szám egész; mindkettő lebegőpontos; egyik ilyen, másik olyan.
- `addCommutative` : két szám összeadva a megadott sorrendben és ahhoz képest fordítva ugyanazt az eredményt adja.

Megengedett egyszerűsítés.

- Egy metódusba szabad egynél több `assertX` hívást is írni.
- Ez realisztikus kódban nem jó választás.
- Létezik erre jobb megoldás (`assertAll`), de az túlmutat a kurzus keretein.

A fentieket a következő módokon lehet bővíteni.

- Néhány (vagy akár mindegyik) feltétel több paraméterrel is kipróbálható (`@ParameterizedTest`).
- A `wrongInput` teszt kerüljön át az `AdderInvalidTest.java` fájlba. Egy test suite hívja meg mindkét osztályt.

Gyakorló feladatok

1. gyakorló feladat

Implementálja a Caesar-kódolást a `cipher.CaesarCipher` osztályban a következőek szerint.

- Az eltolási távolságot konstruktor paraméterként kapja meg (`shift`) és tárolja el.
- Az `encrypt()` metódus egy szöveget kap meg, és az alábbiak szerint, karakterenként kódolja el.
  - Ha a karakter nem betű (nem `'a'` és `'z'` közötti karakter), akkor megtartjuk. Különben a `shift` pozícióval odébb levő karaktert adjuk a kimenethez.
    - Például: `'d'` hárommal eltolva `'g'` lesz.
    - Figyelem: az eltolás átfordulhat: `'z'` kettő távolsággal eltolva `'b'` lesz.
    - Figyelem: az eltolás lehet negatív, és akkor is átfordulhat: `'c'` `-5` távolsággal eltolva `'x'` lesz.
  - További részletek:
    - Az eredmény szöveget `+` operátorral összefűzve is elő lehet állítani, de még jobb `StringBuilder` használatával.

    - Az eltolás után típuskényszerítést kell használni: `(char)shiftedChar` .
- A `decrypt()` metódus legyen az `encrypt()` ellentéte.

A `cipher.CaesarCipherTest` osztály a következőket tesztelje.

- `noShift` : ha `shift` értéke `0` , `encrypt()` és `decrypt()` egyaránt az eredeti szöveget adja
- `encryptBy` : adott szöveg elkódolása adott `shift` távolsággal
  - üres szöveg (a `textBlock` részre `''` alakban írható le)
  - egybetűs szöveg
  - hosszabb szöveg
  - nem csak kisbetűket tartalmazó szöveg
- `inverseOperation` : két, egymással ellentétes hatású művelet meghívása egymás után, az eredmény az eredeti szöveg
  - `encrypt()` , aztán `decrypt()` ugyanolyan `shift` távolsággal
  - `decrypt()` , aztán `encrypt()` ugyanolyan `shift` távolsággal
  - `encrypt()` `shift` távolsággal, aztán `encrypt()` `-shift` távolsággal
  - `encrypt()` `-shift` távolsággal, aztán `encrypt()` `shift` távolsággal
  - `decrypt()` `shift` távolsággal, aztán `decrypt()` `-shift` távolsággal
  - `decrypt()` `-shift` távolsággal, aztán `decrypt()` `shift` távolsággal

2. feladat

a

Írjunk `static math.operation.Power.power()` metódust, amely természetes számok hatványozását valósítja meg (a hatványalap és kitevő is természetes szám). Bár az eredmény mindig egész lesz, a visszatérési érték legyen `double` , mert nemsokára továbbfejlesztjük ezt a függvényt. A `0` nulladik hatványa most legyen `1` .

Tesztelje a `power()` függvényt.

b

Fejlesszük tovább a `power()` függvényt úgy, hogy negatív egész kitevővel is helyesen működjön.

Próbálja ki, hogy az **a** feladatban megírt unit tesztek továbbra is sikeresen lefutnak-e.

Bővítse a unit teszteket negatív kitevő helyességét ellenőrző esetekkel.

06. Tömb, enum, memmap

1. feladat

Készítsük el az `ArrayUtilStructureTest` osztályban leírt szerkezetű programot. Ez egy tömb maximális elemét számítja ki az alábbi módokon. A visszatérési érték mindegyik esetben ugyanaz, a kiszámítás módja azonban eltérő.

- `max()` : a tömb elemeit indexelve járjuk be a tömböt, az eddig érintett rész maximumát számon tartva, és lecserélve, ha az indexelt tömbelem ennél nagyobb
  - Üres tömb maximuma legyen `0` .
  - Nemüres tömbre az érintett rész maximuma az `Integer.MIN_VALUE` értékről induljon.
  - A csere egy egyszerű `if` vizsgálattal történjen.
- `max2()` : hasonlóan, de a csere a `?:` operátorral történjen
- `max3()` : hasonlóan, de a csere a `Math.max()` hívással történjen
- `max4()` : hasonlóan, de a ciklus ne indexelő, hanem bejáró legyen

Írjuk meg az alábbi funkcionális teszteket.

- `maxLength0` : üres tömb maximuma `0`
  - Mind a négy metódust próbáljuk ki.
  - A fentebb említett “Megengedett egyszerűsítés” továbbra is él.
- `maxLength1` : egyelemű tömböt tesztel
  - Paraméterezett tesztelővel próbáljuk ki, ami az egyetlen elem értékét több módon állítja be.
  - Az értékek között szerepeljen `0` , `1` , nagy értékek, negatív értékek.
- `maxLength2` : kételemű tömböt tesztel
  - Paraméterezett tesztelővel próbáljuk ki, ami egy `min` és egy `max` értéket használ.
  - Mind a négy metódust próbáljuk ki úgy is, hogy az értékek `min, max` sorrendben szerepelnek a tömbben, és úgy is, hogy a sorrendjük `max, min` .

A `minMax()` metódus a tömb minimumát és maximumát egy kételemű tömbben adja ki.

- A megvalósítás a maximum/minimum meghatározására a fenti lehetőségek közül bármelyiket használhatja.
- Tipp: szükség lesz az `Integer.MAX_VALUE` értékre.
- Készüljenek ehhez funkcionális tesztek: `minMaxLengthN` , ahol `N=0, 1, 2` .

A `array.util.main.Main.main()` metódus próbálja ki a kódot úgy, hogy a felhasználó kézzel adja meg az adatokat.

- Először az `arrayLenTxt` változóba olvassuk be a tömb hosszát, és alakítsuk számmá az `arrayLen` változóba.
- Ezután készítsünk egy `arrayLen` méretű tömböt.
- Utána olvassunk be `arrayLen` bemenetet, és tároljuk el számként a tömb elemeiben.
- Végül írjuk ki a tömböt magát és a fenti metódusok segítségével meghatározott minimumát/maximumát.
  - A tömbök `System.out.println(array)` módon kiírva nem mutatják meg az

elemeiket…
  - … ezért `System.out.println(Arrays.toString(array))` használatával lehet őket kiíratni, ahol az osztály teljes neve `java.util.Arrays` .
  - A `minMax()` adta tömböt is így írjuk ki.

2. feladat

Készítsük el a `SectorTimerTestSuite` osztályban leírt szerkezetű programokat. Ezek egy versenyautó köreinek időmérését próbálják megvalósítani, az utolsó kivételével szándékosan rosszul.

Először a `race.car.WrongSectorTimer1` osztályt készítsük el. *Szándékosan rontsuk el úgy*, hogy az adattagjai `public` láthatóságúak legyenek. Próbáljuk ki a következőket.

- A következő tesztesetek mind úgy indulnak, hogy egy `1, 2, 3` tartalmú tömbbel példányosítani kell az osztályt.
  - Kísérletező kedvűeknek: `@BeforeEach` használatával még elegánsabb megoldás adható.
- `seemsGood` : az adattag három elemű tömb a megfelelő értékekkel
- `setArrayElemsBreaksEncapsulation` : a tömb mező egyes értékeit közvetlenül el lehet érni és be lehet állítani; ez a hosszát nem változtatja meg
  - Ez a teszt (és a következő) szokatlan, mert egy **direkt rossz** megvalósítást próbál ki.
  - Éppen ezért akkor zöld (elfogadott), ha **rosszul működik az osztály**.
- `setArrayElemsBreaksEncapsulation` : az adattagban a tömb teljesen lecserélhető, ez megváltoztatja a hosszát és az értékeit is

Próbáljuk megjavítani: készítsük el a `race.car.WrongSectorTimer2` osztályt. Az adattag láthatósága legyen most `private` , de a getterek, setterek és a konstruktor megvalósítása legyen naívan egyszerű.

- Most már nem elérhető kívülről az adattag. Ez helyes.
- `seemsGood` : mint korábban
- `constructorBreaksEncapsulation` : a `WrongSectorTimer2` példányt egy lokális változóban levő, `1, 2, 3` tartalmú tömb inicializálja, majd írjuk a tömb elemeit
  - Ez a `timer.getSectorTimes()` eredményének hosszát nem változtatja meg.
  - Ellenőrzendő: a `timer.getSectorTimes()` és `timer.getSectorTime()` eredményeként viszont így más eredmény adódik.
- `getterBreaksEncapsulation` : írjuk a `timer.getSectorTimes()[n]` értékeket
  - Ez a `timer.getSectorTimes()` eredményének hosszát nem változtatja meg, de az elemeket igen.
- `setterBreaksEncapsulation` : a setternek egy `4, 5, 6, 7` tartalmú, lokális változóban tárolt tömb átadása, majd a változón keresztül a tömbelemek módosítása
  - Ez a reprezentáció tömb hosszát és értékeit is módosítja.

Most tényleg javítsuk meg az enkapszulációt: `race.car.SectorTimer` osztály.

- A konstruktor, getter, setter egyaránt készítsen másolatot annak során, hogy átveszi/kiadja a reprezentáció tömbjét.
  - Még jobb, ha a konstruktor és a setter egy közös, privát `initSectorTimes` segédfüggvényre hív át.
- A tesztelőben legyen `constructorEncapsulatesWell`, `getterEncapsulatesWell` és `setterEncapsulatesWell` metódus.
  - Ezek ugyanazt próbálják ki, mint a fentiek, és ugyanazokat az `assertX` vizsgálatokat hajtsák végre.
  - Figyeljük meg, hogy az enkapszuláció most jól működik: nem tudunk kívülről „betörni” a reprezentációba.
- A konstruktornak/getternek/setternek legyen olyan változata is, ami a tömb másolását a `System.arraycopy` művelettel hajtja végre.
  - Célszerű aSectorelv: ne találjuk fel a spanyolviaszt, ha elérhető beépített/könyvtári/sztenderd megoldás/eszköz.
  - Mivel egy osztályban nem lehet két, azonos paraméterezésű konstruktor, az egyik kapjon egy extra `boolean` paramétert, amit nem használunk benne.
  - A konstruktor és a setter *vararg* stílusban kapja meg a paramétereit, ne csak egyszerű tömbként.

3. feladat

Készítsük el az `ElectionTestSuite` által leírt osztály és felsorolási típust. Egy olyan szavazást írnak le, amelyben jelöltekre lehet szavazatokat leadni.

Az `ElectionTestStructureTest` osztály az elkészítendő tesztek szerkezetét mutatja.

- `noVotes`: ha nincsen egy szavazat sem, az első jelölt győz (ez most `JACK`)
- `singleCandidate`: a megadott számú szavazatot adják le, mindegyiket ugyanarra a jelöltre, aki ezáltal győz
  - Felsorolási típus átadható a tesztelőnek: a metódus paraméterének legyen `Candidate` a típusa, és a CSV részre az enum-érték neve kerüljön.
- `twoCandidatesSameVoteCount`: ketten ugyanannyi szavazatot kapnak, aki az enumban a korábbi elem, az győz
  - Kipróbálandó: bármelyik sorrendben is kapják a szavazatokat, ugyanez az eredmény.
- `candidateVoteCount`: a jelöltek sorban 1, 4, 3 és 2 szavazatot kapnak
  - Kipróbálandó: `getCandidatesWithMoreVotesThan` a helyes eredményt adja a 0, 1, 2, 3 és 4 paraméterre is.
  - Mivel egy `@CsvSource` nem tud nyilvánvaló módon tömböt kapni paraméterként, a hívásokat és az elvárt értékeket sorrendben bele kell írni a metódusba.
    - Nehezebb változat: az elvárt értékeket egy szövegben (így: `'JILL,SAM,MAX'`) kell átadni, és ezekből egy rövid kódrészlet alakítja ki az elvárandó `Candidate` tömböt.
  - Itt nem `assertEquals`, hanem `assertArrayEquals` vizsgálat szükséges.

4. feladat

Rajzoljon memóriatérképet (memory map) a következő Java program kommentben jelzett soraihoz. (Másképp: Rajzolja fel a stack és heap pillanatnyi állapotát következő Java program végrehajtása során). A konstruktor paramétereitől tekintsünk el.

Az (5) végrehajtása után mely objektumokat törölheti a szemétgyűjtő?

Main.java:

```
class Foo {
    private int x;

    public Foo(int initX) {
        x = initX;
    }
}

public class Main {
    public static void main(String[] args) {
        int counter = 10;        // (1)

        Foo obj;                 // (2)
        obj = new Foo(5);        // (3)

        Foo obj2 = new Foo(7);   // (4)
        obj2 = obj;              // (5)

        // ...
    }
}
```

Írja át `Foo` konstruktorát hogy `initX` helyett `x` legyen a paraméter, tegye egyértelművé a `this` kulcsszóval, hogy melyik `x` azonosítóra hivatkozik.

Gyakorló feladatok

1. gyakorló feladat

Rajzoljon memóriatérképet (memory map) a következő Java programhoz. (Másképp: Rajzolja fel a stack és heap pillanatnyi állapotát következő Java program végrehajtása során).

```
class Foo {
    private int x;

    public Foo(int x) {
        this.x = x;
    }
}

public class Main {
    public static void main(String[] args) {
        Foo obj = new Foo(0);
        obj = new Foo(10);
        Foo obj2 = obj;
        int i = 1;
        obj2 = new Foo(20);
    }
}
```

07. I/O, kivétel

Feladatok

1. feladat

A `SingleLineFileStructureTest` alapján írjuk meg a `text.to.numbers.SingleLineFile.addNumbers()` metódust, ami egy fájlnevet vesz át, és az első sorában található számok összegével tér vissza.

- Ha nem létezik a fájl, az `IOException` kivétel terjedjen ki a metóduson kívülre.
- Ha a fájl üres, váltsunk ki `IllegalArgumentException` kivételt `Empty file` üzenettel.
- A fájl első sorának minden olyan szavát, ami nem értelmezhető `int` értékként, írjuk ki a `System.err` hibafolyamra. Erre ugyanúgy lehet írni, mint a `System.out` folyamra.
- A fájl használatának a végén erőforráskezelő try eszközzel garantáljuk a fájl bezárását.

Készítsük el az alábbi teszteseteket.

- A két kivételes eset tesztelése.
  - A tesztelő metódus egyszerűen kapja el a kivételeket.
    - Ha mégse váltódna ki kivétel, hívjuk meg a `fail()` metódust. Ez kaphat egy hibaüzenetet is.
    - A második esetben a kivétel szövege így kérhető le: `e.getMessage()`.

  - Mj.: a JUnit `assertThrows()` metódusával még elegánsabb megoldás adható, de ez egy haladóbb szintű eszköz, nem célunk használni.
- Egy érvényes fájl tesztelése, melynek első sora `1 2 not text 3 -123` tartalmú.
  - A kimenetre kiírt szövegeket nem teszteljük JUnit segítségével, de ott kell lenniük.
- Mj.: a fentiek nem valódi egységtesztek, mert a fájlrendszert is használják.

2. feladat

A `MultiLineFileStructureTest` alapján írjuk meg az `text.to.numbers.MultiLineFile.addNumbers()` metódust. Ez a fájl összes sorának összes számát adja meg. Ennek legyen még egy paramétere, az elválasztójel karaktere.

Ezt is teszteljük. Az előzőhöz képest különbség: üres fájl esetén nem váltódik ki kivétel. Az érvényes fájlt teszteljük szóköz és vessző elválasztójellel egyaránt.

3. feladat

Az előző két megoldás ne csak a hibafolyamra, hanem a `wrong.in.txt` fájlba is írja ki a hibás szövegeket (ha a bemeneti fájl neve `in.txt`).

Ehhez nem kell JUnit tesztelőt írni.

4. feladat

A `StatisticsStructureTest` alapján írjuk meg a `textfile.Statistics.numberOfLines()` metódust. Ez kapjon meg egy fájlnevet, és adja vissza, hány sorból áll. Ha a fájl nem létezik, a visszatérési érték legyen `0`.

Ugyanebben az osztályban `numberOfCharacters()` adja meg egy fájlról, hány karaktert tartalmaz (sorvége jelek nélkül).

Mindkettő tesztelendő.

5. feladat

A `FileContentStructureTest` szerint írjuk meg a `textfile.lookup.FileContent.contentLineCount()` metódust. Ez egy fájlnevet és egy szöveget kap meg, és megadja, a fájl hány sorában található meg ez a szöveg.

- Tipp: a `String` osztály dokumentációjában található egy hasznos metódus ehhez.

A `textfile.lookup.FileContent.contentLineNumbers()` metódus szintén egy fájlnevet és egy szöveget kap meg, és egy tömbben visszaadja az összes olyan sor sorszámát (egytől számozva), amelyben a szöveg megtalálható.

Mindkettő tesztelendő.

Gyakorló feladatok

1. feladat

Készítsük el a `plane.util.CircleUtil.writeCircle()` metódust, ami egy fájlnevet és egy `plane.Circle` objektumot vesz át. Ez nyissa meg a fájlt, és írja bele a kör adatait a következőképpen:

```
12.34
-54.12
54.3
```

Készítsük el a `plane.util.CircleUtil.readCircle()` metódust, ami egy fájlnevet vesz át. A metódus a fájlból a fenti formátum szerint olvassa fel egy kör adatait, és ebből elkészít és visszaad egy `Circle` objektumot. Feltehető, hogy a fájl létezik, és a megfelelő módon tartalmazza az adatokat.

Készítsünk tesztelőt, amely egy kör adatait beleírja egy fájlba, majd visszaolvassa azokat, és megvizsgálja mindegyik adattagot.

- Az `assertNotSame()` meghívásával az is ellenőrizhető, hogy az eredeti kör és a beolvasott különböző objektumok.

2. gyakorló feladat

A `file.line.SumChecker.main()` kódja nyissa meg olvasásra a `sum_input.txt` fájlt, írásra a `sum_output.txt` fájlt. Az előbbi tartalma ilyen:

```
7 2,5,-7,6,9
-2 2,5,-7,6,9
12 2,5,-7,6,9
12 2,6,5,-7,6,9
```

Készítsünk kódot, ami soronként megkeresi, hogy az első szám előáll-e a felsoroltak közül két szám összegeként.

- Két, egymásba ágyazott ciklussal kell bejárni a számokat.
- Ugyanazt a számot nem szabad kétszer felhasználni: a harmadik sorban nem `12=6+6` az eredmény.

A kimenetbe ez kerüljön:

```
7=2+5
-2=5+-7
12 none
12=6+6
```

Tipp: az alábbi `continue` utasítás hasznos lehet:

```
outer:
ciklus (...) {
    ciklus (...) {
        ...
        continue outer;
    }
}
```

08. `List`, `Set`, `Map`

Demo

1. demó

Készítsük el a `WorkerScheduleStructureTest` szerint a megadott osztályt. Ennek adattagja leírja, hogy melyik héten kik dolgoznak. A heteket a hívó fél `1` kezdéssel számozza, de belül `0` kezdéssel tároljuk el.

Dolgozókat két `add()` művelettel adunk a munkarendhez.

- Az egyik egy specifikus héthez adja hozzá a második paraméterben megadott dolgozókat.
  - Hozzáadhatók egyenként is, de a `HashSet` osztály `addAll()` metódusával is.
  - Ha a hét még nem szerepel a `weekToWorkers` adattagban, be kell tenni dolgozók nélkül (üres halmazt hozzárendelve).
- A másik több hetet add meg, a dolgozókat mindegyikhez hozzá kell adni.
  - Ez a metódus hívjon át az előzőre.

A másik két metódus megadja, hogy egy adott dolgozó szerepel-e a munkarend egy megadott napján, illetve megadja az összes olyan napot, amelyeken egy adott dolgozó dolgozik.

- Az utóbbihoz használjuk a `HashMap` osztály `entrySet()` metódusát a kulcs-érték párok bejárásához.
  - Ez `Entry<kulcs típusa, érték típusa>` értékeket jár be (`java.util.Map.Entry`).

A következő módon tesztelendő.

- `emptySchedule`: üres munkarendben senki nem dolgozik semelyik napon
  - Itt `assertFalse(<feltétel>)` is használható, ami rövidebb, mint a szintén érvényes `assertEquals(false, <feltétel>)`.

- `schedule` : vegyünk fel néhány dolgozót mindkét `add()` használatával, és aztán próbáljuk ki a két másik metódust.
  - A második változathoz néhány `ArrayList` re van szüksége. Ezek összeállíthatók úgy, hogy minden névre meghívjuk az `add` metódust.
  - Összeállítható így is: `new ArrayList<>(List.of(name1, name2, ...))` . Ebben `List` csomagja szintén `java.util` .
  - A teszt működtethető a szokásos módon `@ParameterizedTest` segítségével is, amivel rövidebb és átfogóbb megoldás adható.

Feladatok

1a. feladat

A következő feladatok kódját a `ListUtilStructureTest` szerint kell elkészíteni.

A `divisors()` metódus a paraméterként kapott nemnegatív egész szám osztóit adja vissza egy `ArrayList` adatszerkezetben.

Tesztelendő `0` , `1` és `64` bemenetekre.

1b. feladat

A `withSameStartEnd()` metódus szövegeket kap meg `ArrayList` adatszerkezetben, és ezek közül azokat adja vissza egy másik `ArrayList` ben, amelyek nem üresek, nem is `null` értékek, és első és utolsó karakterük megegyezik.

A tesztelés egy üres `ArrayList` tel induljon. Erre a `withSameStartEnd()` hívásnak üres listát kell adnia. Ezután ugyanebben a metódusban sorban adjuk a listához az alábbi szövegeket; minden hozzáadás után tesztelendő, mit ad ki a hívás.

- üres szöveg (a kimenet nem változik meg)
- `null` (a kimenet nem változik meg)
- egyetlen szóközt tartozó szöveg (belekerül a kimenetbe)
- az `x` szöveg (belekerül a kimenetbe)
- üres szöveg (a kimenet nem változik meg)
- a `different start and end?` szöveg (a kimenet nem változik meg)
- az `ends and starts the same` szöveg (belekerül a kimenetbe)

1c. feladat

A `maxToFront()` metódus a megkapott lista ábécérendben utolsó elemét helyezze át a lista elejére.

- A listát helyben kell megváltoztatni, ne jöjjön létre új lista.

- Ha a lista `null` vagy üres, nincsen teendő.
- Tipp: `Collections.max()` .
- Tipp: az `ArrayList` osztály megfelelő metódusaival el lehet távolítani a listából a megtalált értéket, majd vissza lehet tenni a `0` indexre.

A következő bemenetek tesztelendők.

- `null` : `assertX` jellegű hívásra nincsen szükség, csak meg kell hívni a metódust
- üres lista
- egyelemű lista
- három szöveget tartalmazó lista: `can, you, succeed` , erre az elvárt kimenet `you, can, succeed`
- három szöveget tartalmazó lista: `-123, 2000, 100` , erre az elvárt kimenet `2000, 100, -123`

4. feladat

A `CharacterStatisticsStructureTest` osztály alapján dolgozzunk.

Az osztály konstruktora egy szöveget kap meg, és az ebben szereplő összes karakterre beteszi annak előfordulási gyakoriságát az adattagba.

- Tipp: a `String` osztály `toCharArray()` metódusa segítségével könnyen elérhetők a szöveg karakterei.
- Tipp: a `HashMap` osztály `getOrDefault()` adattagja is jól használható itt. Oldjuk meg a feladatot a használatával és a használata nélkül is.

A `getCount()` metódus az adott karakter gyakoriságát adja meg.

Szintén elkészítendő egy `toString()` implementáció az osztályhoz, ami a karakterek gyakoriságának szöveges reprezentációját mutatja. A metódus kapja meg az `@Override` annotációt. A gyakoriságok az `entrySet()` sorrendjében jelenjenek meg benne.

Tesztelés: az alábbi szövegekre mind vizsgáljuk meg legalább egy karakter gyakoriságát.

- Kipróbálandó az az eset is, amikor a megadott karakter nem szerepel a szövegben.

A `toString()` kimenete szintén megvizsgálandó.

- A kimenetek az alábbiakhoz hasonlóak lesznek, de a karakterek sorrendje függhet a rendszertől.
- A kimenetben a szóközöknek is betűre pontosan egyezniük kell.

A szövegek a következők:

- üres szöveg: üres kimenet
- `aaaaaaaa` : `a(8)`
- `HgFeDcBa` : `a(1) B(1) c(1) D(1) e(1) F(1) g(1) H(1)`

- `a?!_#@{}` : `@(1) a(1) !(1) #(1) {(1) }(1) ?(1) _(1)`
- `Hello world!` : `(1) !(1) r(1) d(1) e(1) w(1) H(1) l(3) o(2)`
  - Itt az első karakter a szóköz ( ).

09. Sablon

Feladatok

1. feladat

Készítsen a `MultiSetStructureTest` szerint olyan osztályt, amely `E` típusú elemek multiplicitásos halmazát reprezenzálja. Ennek adattagja leírja, hogy milyen elemből hány darab van a halmazban. Ezt az adatszerkezetet zsáknak (bag) is szokták nevezni.

Az osztály konstruktora a paraméterül kapott elemekkel tölti fel kezdetben a zsákot.

Elemeket az `add()` művelettel adhatunk a zsákhoz. Ha az elem még nem volt benne a zsákban, kerüljön bele `1` multiplictással, ha pedig már benne volt, akkor nőjön meg eggyel a multiplicitása. A visszatérési érték a megnövelt multiplicitás legyen.

A `getCount()` a paraméter multiplicitását kérdezi le. Ha ez nem szerepel a zsákban, legyen `0` az eredmény.

Az `intersect()` két zsák metszetét állítja elő. Ebben azok az elemek lesznek benne, amelyek mindkettőben megtalálhatóak, ennek a multiplicitása a kisebbik. A metódus egyik eredeti zsákot se változtassa meg, az eredmény egy új `MultiSet` objektum legyen.

A `countExcept()` megadja a zsákban található összes elem darabszámát (multiplicitással együtt), de a paraméter elemeit nem veszi figyelembe.

A következő módon tesztelendő:

- `multiSetInteger` , `multiSetString` : a névben megadott típusú zsák
  - Adjunk hozzá pár elemet az `add()` metódussal, illetve a konstruktorban, és vizsgáljuk meg a `getCount()` eredményét.
- `intersect` , `countExcept` : hozzunk létre két zsákot, és próbáljuk ki rajtuk az `intersect()` és `countExcept()` függvényeket

2. feladat

Az `OrganiserStructureTest` alapján készítsünk olyan osztályt, amely adatokat tárol sorban az `elems` adattagjában. A másik adattag pozíciópárokat (cseréket) tartalmaz.

A konstruktor a megkapott adatok alapján feltölti az `elems` adattagot. Kezdetben cserék nincsenek beállítva.

- A konstruktor kódjára figyelmeztetést fog adni a fordítóprogram. Mivel a megvalósításban most biztosan nem fog veszélyes kód szerepelni, a konstruktor megkaphatja a `@SafeVarargs` annotációt.

A `get()` metódusnak legyen kétféle paraméterezése.

- Az egyik az `elems` egy érvényes indexét veszi át, és visszaadja az indexelt elemet. Feltételezhető, hogy a paraméter érvényes.
- A másik változat mindegyik eltárolt elemet visszaadja egy listában.
  - Vigyázzunk, hogy ne szivárogjon ki az osztály belső állapota!

Az `addSwap()` művelet egy egész számokból álló párt ad a `swaps` adattaghoz.

- Mivel a Javának nincsen beépített pár/rendezett n-es adatszerkezete, az adattag `Map.Entry` típusú elemekből áll.
  - Ez `Integer` típusparamétereket kap, mert típusparaméterek nem lehetnek primitívek.
- A Map.entry(from, to) hívással készíthető ilyen "pár".

A `swap()` segédmetódus cserélje meg a két megadott indexű elemet az `elems` adattagban.

- Ez a metódus ne látszódjon az osztályon kívül.
- Tipp: átmeneti változó tárolja el az egyik értéket.

A `runSwaps()` sorban végrehajtja a `swaps` listában leírt cseréket.

- Az egyes cserék végrehajtásához a `swap()` hívandó meg.

Az osztály szöveges reprezentációja ilyen alakú legyen: `[1 3 5 7 9 ]` vagy `[d a c b e ]`.

3. feladat

A `RangedStackSuite` alapján készítsünk vermet ábrázoló osztályt, amelybe egyszerre több elem tehető be/vehető ki.

Lehessen üresen is elkészíteni, és másoló konstruktorral is. Ez utóbbi egy másik `RangedStack` példányhoz hasonló tartalmú vermet készít el, de az adattagjaik ne egy közös listára mutassanak (aliasing).

A következő módon tesztelendő.

- `empty` : készüljön el egy üres `RangedStack` és egy másolata
  - Mindkét esetben: akárhány elemet veszünk ki (pl. `0` , `1` vagy `100` elemet egyszerre), a kimenet üres lista.

- `testString`, `testInteger`: mindkét típusra készüljön el `6` elemből egy `RangedStack` (pl. egész típus esetén a `2`, `4`, `6`, `8`, `10`, `12` elemekből), és legyen egy másolata is
  - Az eredetiből vegyünk ki sorban `1`, majd `2`, majd `3`, majd `100` elemet. Az elvárt kimenetek sorban `[12]`, `[10, 8]`, `[6, 4, 2]` és az üres lista.
  - A másolatból vegyünk ki `100` elemet, majd megint `100` elemet. Az első lépés elvárt kimenete `[12, 10, 8, 6, 4, 2]`, a másodiké egy üres listaí.

10. Öröklődés

Feladatok

1. feladat

Készítsünk a `BookSuite` alapján könyveket.

Az osztály mindkét konstruktora az `initBook` metódust hívja meg. A paraméter nélküli konstruktor a `default...` mezők szerint tölti fel az adattagokat, a másik előbb megvizsgálja a kapott adatokat a `checkInitData` metódus segítségével. A vizsgálat `IllegalArgumentException` kivételt vált ki, ha az `author` szöveg rövidebb két betűnél, a `title` rövidebb négy betűnél, vagy ha `pageCount` nem pozitív szám.

A könyv ára az oldalszámmal megegyező összeg.

Teszteléshez használjuk az alábbi három könyvet.

```
John Steinbeck, Of Mice and Men, 107
Kozsik Tamás, Java programozás, 234
Alan Alexander Milne, Winnie-the-Pooh, 145
```

A könyvek elvárt szöveges alakja az alábbi. A `decode()` metódus ilyen szövegekből alakít ki könyveket, tesztelendő, hogy a visszakapott könyvek egyes adatai megegyeznek-e az eredeti példányéival.

- Tipp: szövegek elejéről és végéről a felesleges szóközök a `strip()` metódussal távolíthatók el.

```
John Steinbeck: Of Mice and Men; 107
Kozsik Tamás: Java programozás; 234
Alan Alexander Milne: Winnie-the-Pooh; 145
```

A `getShortInfo()` rövidített alakban írja le a könyvet. A szerzők neveit iniciálékkal rövidíti, ezt a `getAuthorWithInitials()` valósítsa meg egy `StringBuilder` használatával.

```
J. Steinbeck: Of M; 107
K. Tamás: Java; 234
A. A. Milne: Winn; 145
```

Tesztelendő továbbá, hogy olyan könyveket, amelyek nem mennek át a `checkInitData` vizsgálatain, nem tudunk létrehozni, és `InvalidBookException` kivételt váltanak ki. Tesztelendő, hogy ezek a kivételek tartalmazzák a könyv leendő szerzőjét és címét.

Gyermekosztály: `EBook`

Az `EBook` osztály konstruktora egy `Book` példány mellett a három új mezőnek megfelelő adatokat kap meg, ez utóbbiakat eltárolja.

A könyv ára az oldalszámnál `pdfSize` értékkel több.

Ha a fenti könyvek `pdfSize` értéke rendre 5, 10000 és 8, akkor a szöveges reprezentációik az alábbiak legyenek. Itt a vásárlók szemét megtévesztő évtizedes gyakorlat szerint a valós árnál `0.01` egységgel kevesebb van feltüntetve, ha az ár `1000` egység vagy kevesebb.

```
EBook(John Steinbeck: Of Mice and Men; 107, $111.99)
EBook(Kozsik Tamás: Java programozás; 234, 10233 Ft)
EBook(Alan Alexander Milne: Winnie-the-Pooh; 145, £152.9
```

Ezeket dekódolni nehéz lenne, a `decode()` metódus váltson ki `UnsupportedOperationException` kivételt.

Gyermekosztály: `PrintedBook`

Egy `PrintedBook` a `Book` szokásos adatai mellett kaphasson meg egy `CoverType` értéket, amit tároljon el az adattagba. Egy `PrintedBook` a szokásos könyveknél hat oldallal többel rendelkezik.

Egy `PrintedBook` példányt egy `Book` példány adatai segítségével is elő lehessen állítani. Ez legyen mindig keménykötésű. Ez a konstruktor hívjon át az előzőre.

A könyv ára az oldalszám többszöröse legyen: puhafedelű könyvek esetén kétszerese, keménykötésűek esetén háromszorosa.

Amennyiben a könyveket a második konstruktorral állítjuk elő, a szöveges reprezentációjuk az alábbi alakú legyen.

```
John Steinbeck: Of Mice and Men; 113 (HARDCOVER)
Kozsik Tamás: Java programozás; 240 (HARDCOVER)
Alan Alexander Milne: Winnie-the-Pooh; 151 (HARDCOVER)
```

Ezt könnyű dekódolni, a `decode()` metódus itt legyen megvalósítva.

- Tipp: mivel a zárójelek a `String` osztály `split()` metódusában speciális jelentésűek, így használandó: `txt.split("[(]")`

Könyvgyűjtemény

A `BookCollection` tároljon el sok könyvet egy listában, és ezeket lehessen elmenteni és betölteni az alábbi formátumban.

```
Book--John Steinbeck: Of Mice and Men; 107
Book--abcd: ABCD; 123
EBook--EBook(abcd: ABCD; 123, 356.99$)
PrintedBook--abcd: ABCD; 129 (HARDCOVER)
```

Ha `EBook` vagy érvénytelen adatú könyv érkezne, a helyére egy `DamagedBook` példány kerüljön.

A mentést és töltést próbáljuk ki üres listával, egyetlen (alapértelmezett adatokkal elkészített) könyves listával, és egy mindenfélet (`EBook` és `PrintedBook` könyvet egyaránt tartalmazó) listával.

2. feladat

Készítsünk `StuffyList` adatszerkezetet a megadott struktúra szerint.

Ez az adatszerkezet olyan, mint egy átlagos lista, ami `E` típusú elemek halmazaiból áll… kivéve, hogy az `add()` művelet garantálja, hogy egy újonnan hozzáadott halmaz tartalmazza a `neededElement` elemet.

- Ha a `neededElement` hiányozna a paraméterként kapott halmazból, ne az eredeti halmaz objektumot változtassuk meg, hanem készüljön új halmaz, és ez kerüljön be a listába.

A teszteléshez készüljön néhány `StuffyList` úgy, hogy a megkapott halmazok tartalmazzák/nem tartalmazzák a `neededElement` értéket. Megvizsgálandó, hogy a hozzáadott halmazokban mind szerepel az elvárt érték.

- Az elvárt érték `List.of(...)` és `Set.of(...)` műveletekkel is előállítható.
  - Ezeket a metódusokat csak `assert` jellegű hívások elvárt értékeinél használjuk, **máshol ne**.

11. Interfész, egyenlőség

Feladatok

1. feladat

A `MultiDimensionalPointSuite` alapján készítsünk többdimenziós pont típust. Ennek a konstruktora a kezdeti koordinátákat veszi át. A `get` és `set` metódusokkal egyes koordinátákat lehet beállítani és lekérdezni.

Az osztály két interfészt valósít meg.

- `Scalable` : minden koordinátát a megkapott mértékben növel meg
- `Undoable` : itt összetettebb a teendő
  - Minden olyan metódus működését vissza akarjuk tudni vonni, ami megváltoztatja a pont állapotát.
  - Ezt elősegítendő, a `lastCoordinates` adattag mindig a pont legutóbbi állapotát tartalmazza.
    - A pont elkészítése után közvetlenül a kezdeti koordinátákkal legyen feltöltve.
  - Az interfész metódusa az utolsó állapotot állítja vissza.
    - Ha közvetlenül ezután újra meghívják a műveletet, az éppen lecserélt állapot álljon vissza.

Az alábbi három teszteset mindegyikében a következő lépéseket kell megtenni.

1. Pont elkészítése tetszőleges adatokkal.
2. A megteendő lépés elvégzése, lásd lentebb.
3. Megvizsgálandó mindegyik koordináta elvárt értéke.
4. Visszalépés meghívása.
5. Megvizsgálandó, hogy mindegyik koordináta értéke visszaállt az elkészítés utáni állapotra.
6. Visszalépés meghívása ismét.
7. Megvizsgálandó mindegyik koordináta elvárt értéke.

A következők tesztelendők.

- `testInitUndo` : a pontot csak elkészítjük
- `testSet` : a pont elkészítése után beállítjuk egy koordinátáját
- `testScale` : a pont elkészítése után felskálázzuk

Két pont akkor minősüljön tartalmilag egyenlőnek, ha a koordinátáik megegyeznek. A korábbi állapotuk nem számít.

A `testEquality` metódus tesztelje, hogy két olyan lista egyenlő, amelyben egymással egyenlő pontok szerepelnek.

- Mindkét listához használható a `List.of` művelet.
- A pontok különbözőek legyenek a két listában.

Legyen természetes rendezés is adott a pontokon az alábbiak szerint. Ha valamelyiknek nagyobb a dimenziószáma, az minősül nagyobbnak. Különben az első eltérő koordinátaérték dönt, ha pedig mindegyik koordináta megegyezik, akkor egyenlőek.

A `testOrdering` metódusban kerüljön sok különféle pont egy listába. Erre aztán hívjuk meg a Collections.shuffle, majd a Collections.sort műveletet. Ezután teszteljük, hogy a kijött lista elemei az elvárt, növekvő sorrendbe kerültek.

- Mivel a lista tartalma módosul, az elkészítéséhez *nem* használható a `List.of` művelet.
  - Az elvárt érték elkészítéséhez viszont igen.
- Itt nem szigorúan egységtesztelést végzünk, mert az egyes lefutások során különbözően keverjük össze a lista elemeit.

2. feladat

A `GoAroundStructureTest` szerint készítsünk olyan adatszerkezetet, ami egypár elemet tárol. A bejárója (`GoAroundIteratorStructureTest`) működjön úgy, hogy a tárolt elemeket `roundCount` alkalommal adja elő.

Teszteljük, hogy ha az eltárolt elemek `1, 2, 3, 4` és `roundCount` értéke `2` , akkor a bejáró kimenete `1, 2, 3, 4, 1, 2, 3, 4` . Mindegyik lépés megtétele előtt próbáljuk ki, hogy `hasNext()` értéke `true` , de az utolsó lépés megtétele után már `false` .